가상 면접 사례로 배우는 대규모 시스템 설계 기초 2

6장 광고 클릭 이벤트 집계

Ad Click Event Aggregation

Chris Ohk

utilForever@gmail.com

발표 순서

- 요구사항과 기본 설계
  - 무엇을 집계할지, 어느 정도의 부하를 감당할지 결정한다.
- 집계 알고리즘과 시간
  - 분산 집계, Top-K, 재집계, 윈도와 지연 도착을 살펴본다.
- 정확성 / 확장성 / 운영
  - 중복과 장애를 처리하고, 병목과 집계 오류를 찾아낸다.
- 설계 토론
  - 본문 약 50분, 상황별 토론 3개 약 10분으로 진행한다.

클릭을 집계하는 이유

- 광고주는 기간별 성과를 비교한다
  - 광고별 클릭 수와 인기 광고 순위를 빠르게 조회한다.
  - 대시보드 숫자가 늦거나 바뀌는 이유를 설명할 수 있어야 한다.
- 과금에 연결되는 숫자는 재현 가능해야 한다
  - 같은 클릭을 중복 반영하거나 장애로 누락하면 신뢰가 깨진다.
  - 집계 오류를 발견하면 원본에서 해당 기간을 다시 계산한다.

요구사항과 범위

- 조회 기능
  - 광고별로 지정 기간의 클릭 수를 조회한다.
  - 최근 구간의 Top-K와 국가 / 기기별 필터를 지원한다.
- 정확성과 운영 조건
  - 클릭 식별자 기준으로 중복을 제거하고 지연 이벤트를 보정한다.
  - 입력 폭주와 장애를 견디며 과거 구간을 재집계할 수 있어야 한다.
- 설계 범위
  - 부정 클릭 판정은 입력 정책으로 받아 집계 기준에 반영한다.

규모와 서비스 목표

- 발표용 규모 가정
  - 하루 10억 건, 이벤트당 1 KB, 피크는 평균의 5배로 가정한다.
  - 원본은 30일 보관하고, 입력 로그는 복제본 3개를 둔다.
- 발표용 조회 / 집계 정책
  - 기본 집계 단위는 1분이며, 잠정 결과를 지속적으로 갱신한다.
  - 최소 조회 단위 / 정산 확정 시점 / 지연 허용 범위를 별도로 정한다.

처리량과 저장량 계산

발표 가정: 10억 건/일, 1 KB/건, 피크 5배, 로그 3복제

항목	계산	결과
평균 처리량	10억 ÷ 86,400초	약 11,574건/초
피크 처리량	평균 × 5	약 57,870건/초
하루 원본	10억 × 1,000 B	1 TB/일
30일 원본	1 TB × 30일	논리 용량 30 TB

로그 3복제: 총 기록량 약 3 TB/일. 압축 / 인덱스 / 여유 공간은 별도.

클릭 이벤트의 계약

동일 클릭을 다시 보내도 event_id와 발생 시각은 유지한다.

필드	예시	의미
event_id / ad_id	e-701 / A	클릭 식별자 / 광고 식별자
event_time	12:00:50 UTC	클릭이 발생한 시각
ingest_time	12:01:10 UTC	수집 서버가 기록한 시각
country / device	KR / mobile	필터에 사용할 차원
schema_version	1	입력 형식의 버전

예시에서는 날짜를 생략했다. 실제 시각에는 날짜 / 시간대 정보를 포함한다.

조회 API와 결과의 의미

- 광고별 구간 집계
  - GET /v1/ads/A/clicks?from=12:00&to=12:05
  - 구간은 [시작, 끝)으로 해석하고 시간대는 UTC로 통일한다.
- 인기 광고와 필터
  - GET /v1/ads/top?window=5m&k=100&country=KR
  - 지원하는 필터 조합과 정렬의 동점 처리 규칙을 고정한다.
- 응답의 상태
  - count와 함께 갱신 시각 / 집계 버전 / 잠정 여부를 반환한다.

원본 데이터와 집계 데이터

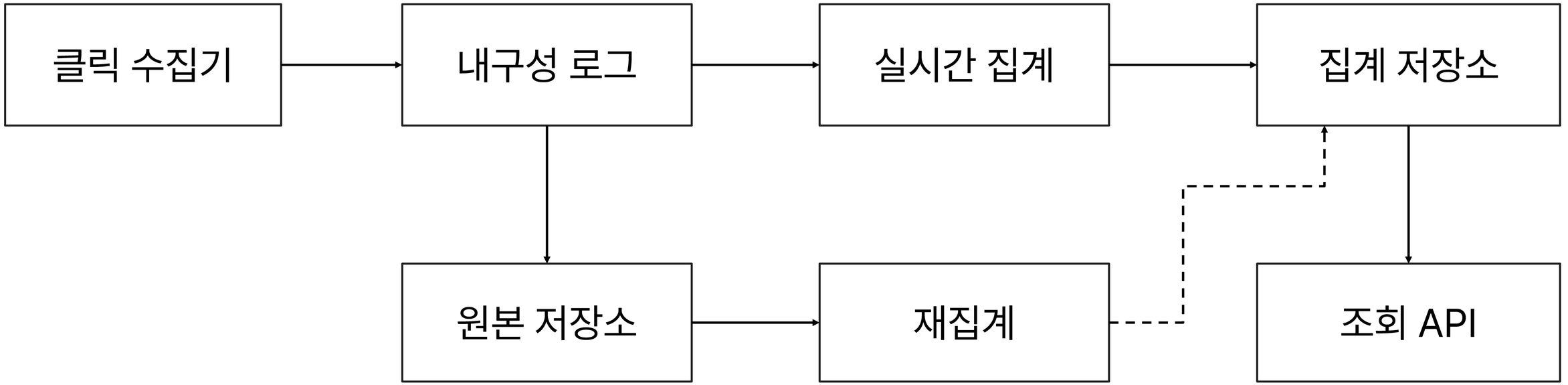
원본은 재계산의 근거이고, 집계값은 빠른 조회를 위한 결과다.

구분	원본 데이터	집계 데이터
단위	클릭 이벤트 1건	광고 / 윈도 / 차원별 합계
예시	e-1, e-2, e-3	A / 12:00 / KR / 3건
읽는 목적	감사 / 검증 / 재집계	대시보드 / 기간별 조회
변경	보존한 입력을 다시 읽음	지연 도착으로 결과 수정

집계값만 남기면 잘못된 규칙을 수정해 과거 결과를 복원하기 어렵다.

전체 데이터 흐름

수집한 이벤트를 보관 경로와 계산 경로에 각각 전달한다.



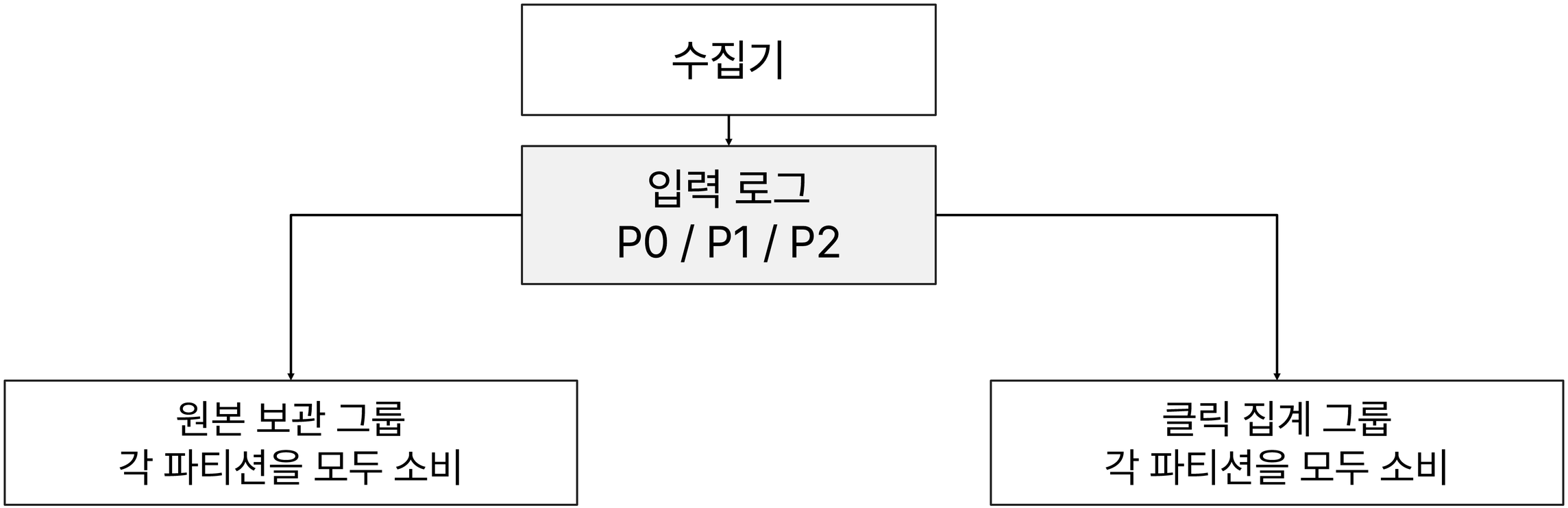
실선: 지속적인 입력과 조회　　점선: 과거 구간의 재계산 결과

수집과 내구성 있는 로그

- 수집 완료의 기준
  - 로그의 내구성 정책을 만족한 뒤 클라이언트에 성공을 응답한다.
  - 응답이 사라진 재시도는 같은 event_id로 다시 보낼 수 있다.
- 집계 속도와 입력 속도의 분리
  - 로그는 짧은 입력 폭주를 흡수하고 장애 후 재생할 데이터를 남긴다.
  - 복제 / 최소 동기 복제본 / 확인 응답 정책을 함께 설정한다.

파티션과 소비자

순서는 파티션 안에서 유지하고, 소비 그룹은 독립적으로 진행한다.



한 소비 그룹 안의 파티션 소유권과 서로 다른 그룹의 읽기를 구별한다.

저장소와 데이터 모델

- 원본 보관
  - 날짜 / 시간 기준으로 나누어 객체 저장소에 순차적으로 기록한다.
  - 원본 스키마와 집계 규칙의 버전을 재집계 작업에 전달한다.
- 집계값 조회
  - (광고, 윈도 시작, 필터 조합)를 결과 키로 사용한다.
  - count, version, 갱신 시각을 저장하고 범위 조회를 최적화한다.
- 저장소 선택
  - 쓰기량 / 범위 조회 / 수정 비용에 맞춰 KV 또는 OLAP 계열을 비교한다.

분산 집계의 계산 구조

Map과 Reduce의 분해 원리를 스트림 연산에도 적용할 수 있다.



입력: 클릭 이벤트

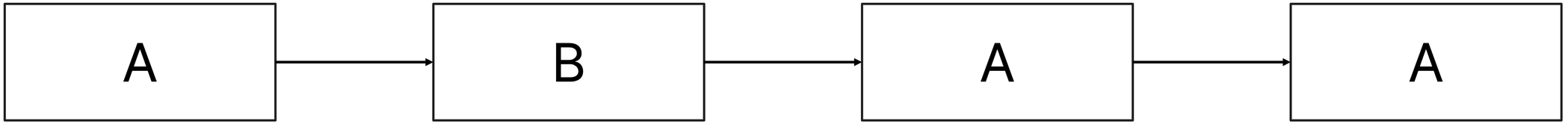
키: (ad_id, window)

값: count

이 흐름을 쓴다는 것과 Hadoop 배치 시스템을 선택한다는 것은 별개의 결정이다.

클릭 수 집계 예제

입력 네 건은 서로 다른 event_id이며 같은 1분 구간에 속한다.



(A, 12:00)
1 + 1 + 1 = 3

(B, 12:00)
1

집계 키가 같으면 같은 상태를 갱신한다. 다음 1분은 별도 키로 구분한다.

시간 단위 확대와 합산

서로 겹치지 않는 작은 버킷을 합쳐 큰 구간을 계산한다.

구간 시작	12:00	12:01	12:02	12:03	12:04
클릭 수	2	4	1	3	5

[12:00, 12:05) = 2 + 4 + 1 + 3 + 5 = 15

고유 사용자 수는 단순 합산할 수 없다. 같은 사용자가 여러 버킷에 나타날 수 있다.

정확한 분산 Top-K

한 광고의 전체 점수가 한 샤드에 있어야 후보 병합이 정확하다.



같은 윈도 / 결과 버전 / 동점 규칙을 적용한다.

부분 Top-K 병합의 반례

각 서버의 Top-1만 보내면 실제 1위인 X를 잃는다.

광고	서버 1	서버 2	전체
A	100	0	100
B	0	100	100
X	99	99	198

서버 1은 A, 서버 2는 B를 보낸다. 부분 count를 광고별로 합친 뒤 순위를 구해야 한다.

필터와 차원의 개수

- 집계 키에 차원이 추가된다
  - (광고, 1분, 국가, 기기)별로 별도의 상태가 생긴다.
  - 활성 광고 수와 실제 차원 조합 수가 상태 크기를 결정한다.
- 필터 조합에는 비용이 있다
  - 자주 쓰는 조합만 미리 집계하고 나머지는 조회 시 계산할 수 있다.
  - 사용자 ID처럼 값 종류가 많은 차원을 무조건 추가하지 않는다.

배치와 스트림 처리

결과를 얼마나 빨리 보여 줄지와 재계산 비용을 함께 비교한다.

방식	계산 경로	운영에서 고려할 점
배치	모은 구간을 한 번에 처리	처리 주기만큼 결과가 늦어짐
스트림	입력마다 상태를 갱신	상태 / 시간 / 복구 관리 필요
Lambda	배치 + 빠른 스트림	두 결과의 병합과 로직 일치
Kappa	로그 재생 + 스트림	재생 비용과 입력 보존 기간

재집계에도 동일한 입력 / 규칙을 적용할 수 있어야 결과를 설명할 수 있다.

과거 데이터 재집계

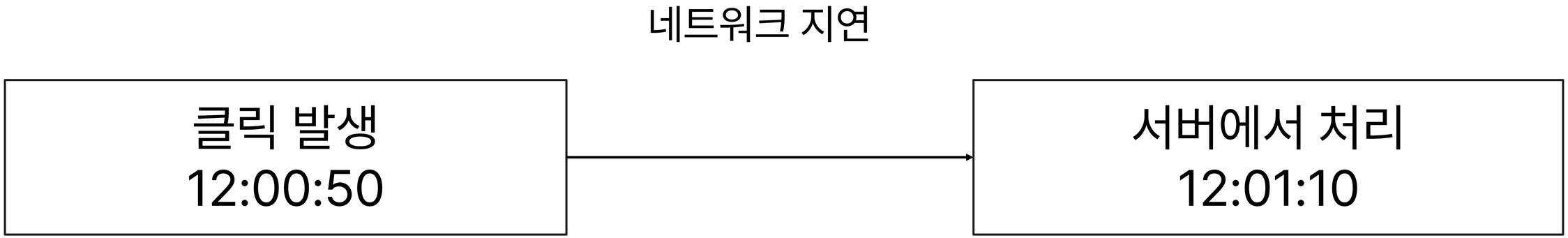
새 결과 버전을 검증한 뒤 조회 대상을 전환한다.



H 이후 입력을 v2에 이어 반영하고, 실패 시 v1으로 되돌릴 수 있게 유지한다.

이벤트 시간과 처리 시간

같은 클릭도 어떤 시각을 기준으로 묶는지에 따라 집계 구간이 달라진다.

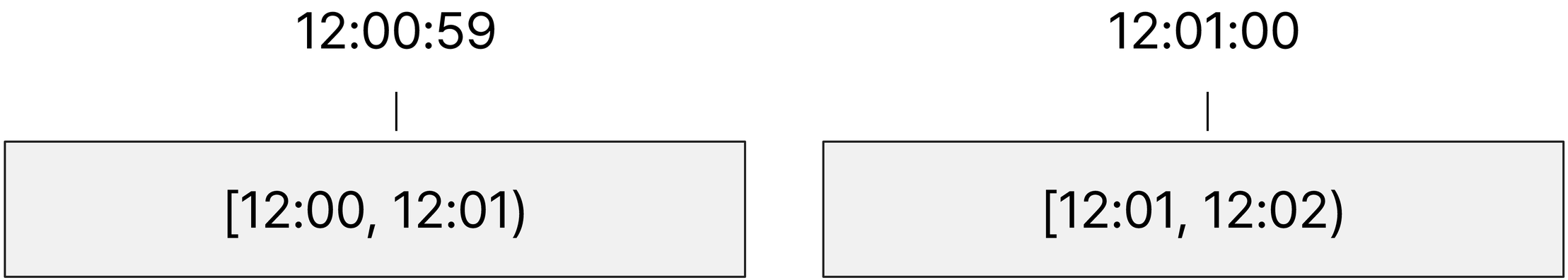


이벤트 시간 집계
[12:00, 12:01)

처리 시간 집계
[12:01, 12:02)

고정 윈도

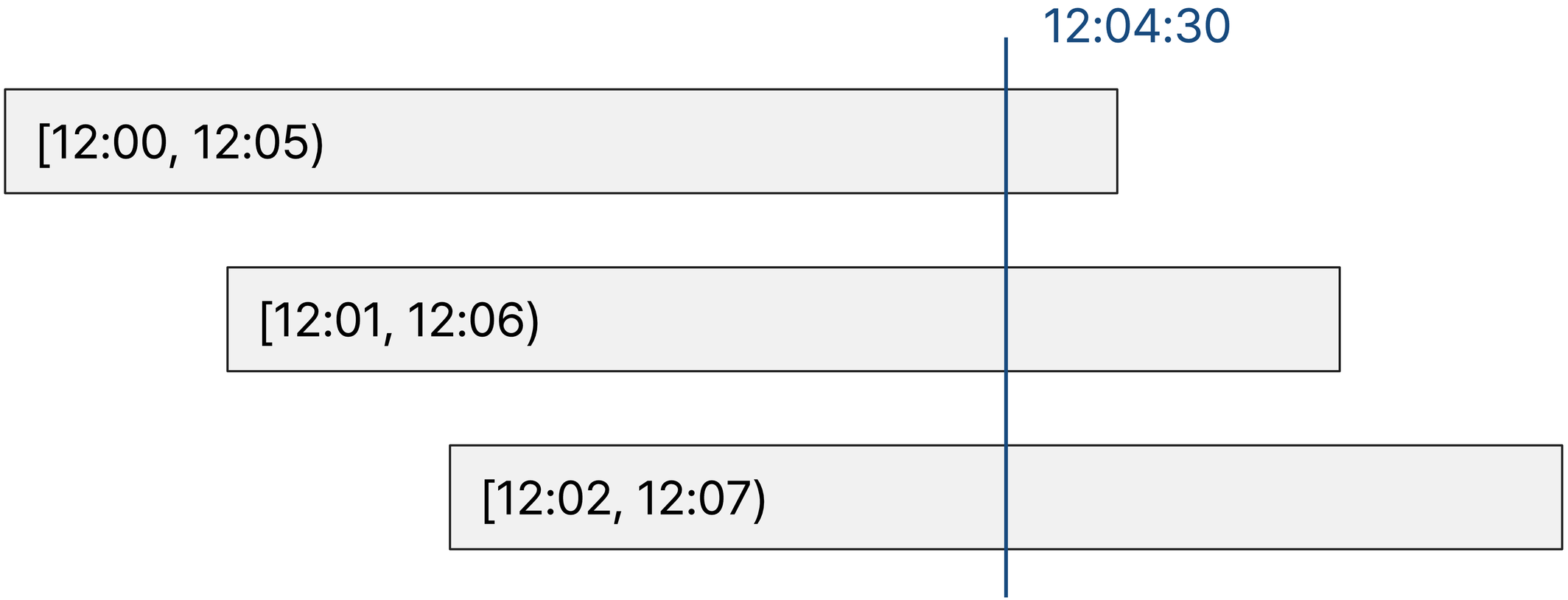
고정 윈도는 겹치지 않는다. 각 이벤트는 한 구간에만 속한다.



경계 규칙: 시작 시각은 포함하고 끝 시각은 제외한다. [start, end)

슬라이딩 윈도

길이 5분, 이동 간격 1분이면 한 이벤트가 여러 창에 속한다.



그림은 겹치는 창 3개만 표시했다. 이 이벤트는 실제로 총 5개의 창에 속한다.

워터마크의 의미

예제 전략: W = 관측한 최대 이벤트 시각 − 20초

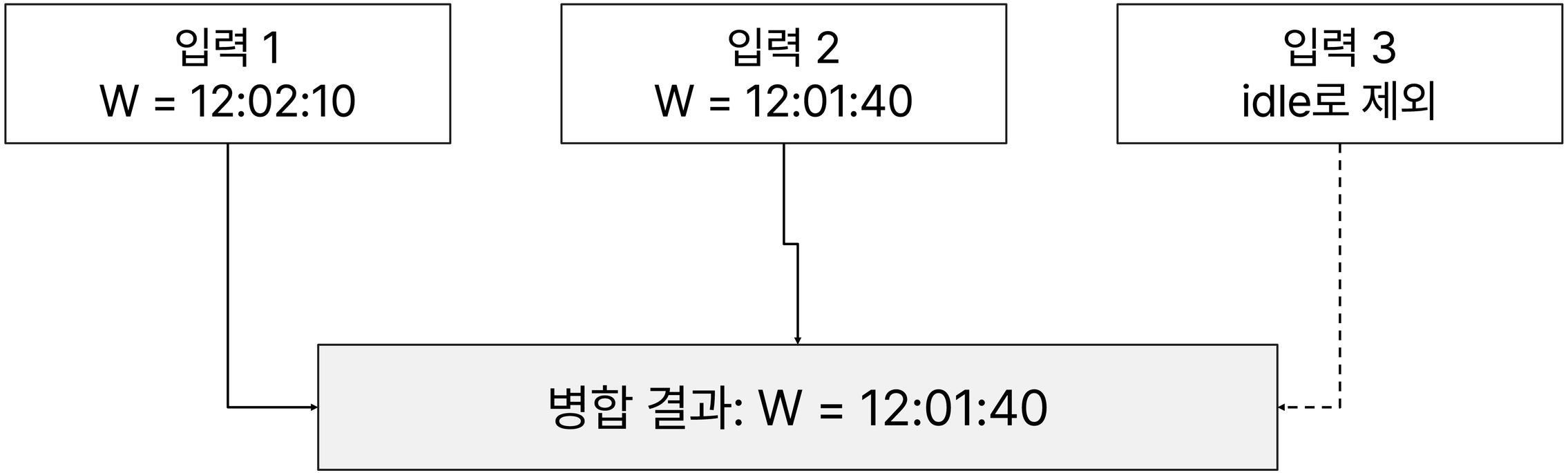
관측한 최대 시각	허용 순서 뒤바뀜	워터마크 W
12:01:30	20초	12:01:10

[12:00, 12:01) 창의 최초 출력 가능

W는 이벤트 시간의 진행 판단이다. 과거 이벤트가 더 오지 않는다는 보장은 아니다.

여러 입력의 워터마크

병합 연산자는 활성 입력들의 워터마크 중 최솟값을 따른다.



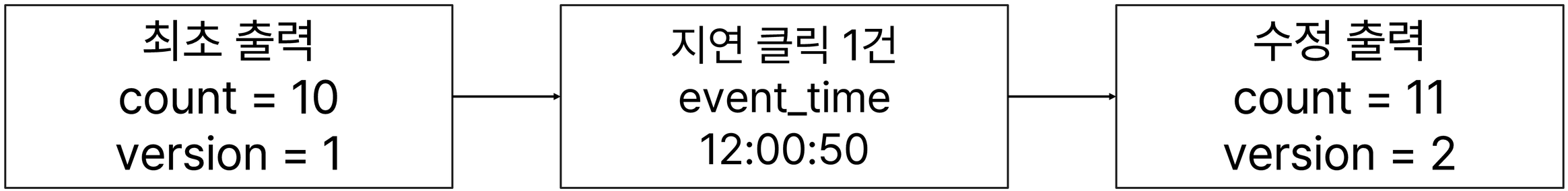
유휴 입력을 제외하는 설정과 재개 시 늦은 이벤트를 다루는 정책이 필요하다.

지연 이벤트 처리 정책

- 최초 결과와 수정 결과
  - 워터마크가 창의 끝을 지나면 최초 결과를 출력할 수 있다.
  - 허용 지연 동안 상태를 유지해 늦게 온 클릭을 반영한다.
- 상태를 정리한 뒤의 이벤트
  - 허용 범위를 넘은 입력은 별도 경로로 보내 재집계 / 보정한다.
  - 대시보드의 잠정치와 업무상 확정값의 기준을 구분한다.

늦은 클릭의 수정 예제

창 [12:00, 12:01), 허용 지연 120초인 개념 예제



현재 W = 12:01:10: 아직 상태를 보존하므로 수정 가능

상태 정리 기준은 W ≈ 12:03:00이다. 수정된 전체 값 11을 기존 10에 더하지 않는다.

결과가 보일 때까지의 지연

- 데이터 신선도는 여러 지연의 합이다
  - 전송 / 로그 대기 / 처리 / 출력 확정 / 조회 갱신 시간이 누적된다.
  - 워터마크 대기와 체크포인트 커밋도 출력 시점에 영향을 준다.
- 빠른 잠정값과 안정된 확정값
  - 조기 출력은 빨라지지만 수정 횟수와 저장소 쓰기량이 늘어난다.
  - 5초 갱신 목표와 10분 늦은 클릭의 완전성은 별도로 설계한다.

전달 보장 비교

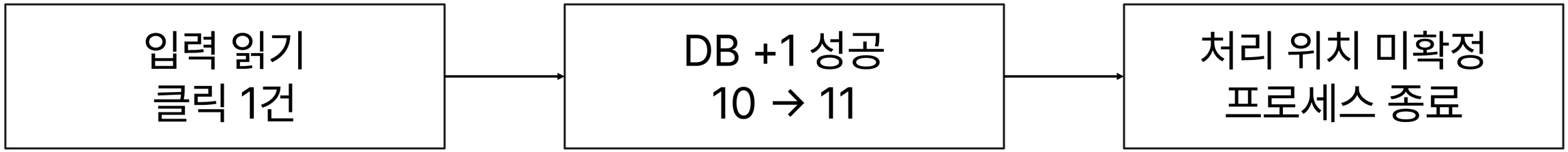
같은 로그 레코드를 다시 처리할 때 어떤 결과가 남는지 비교한다.

보장	누락 가능성	재전달 / 재실행	결과에 필요한 조건
At-most-once	가능	하지 않음	일부 입력의 유실을 허용
At-least-once	재시도로 방지	가능	중복 효과를 별도로 방어
Exactly-once	복구로 방지	가능	관리 상태에 한 번의 효과

외부 출력까지의 Exactly-once에는 재생 가능한 Source와 안전한 Sink 계약이 필요하다.

DB 쓰기 직후 장애

DB 반영과 입력 위치 확정 사이에 장애가 발생한다.



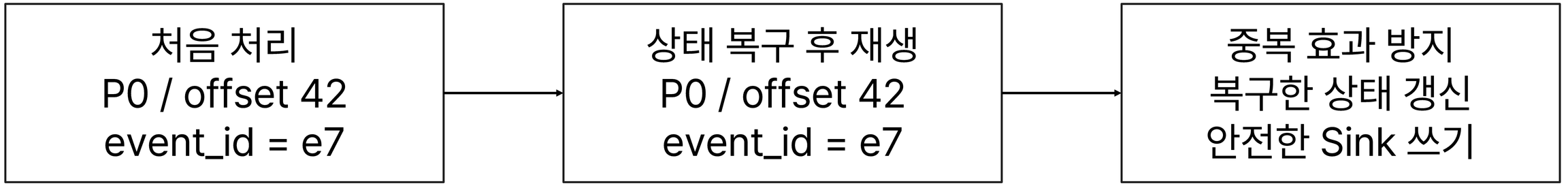
재시작 후 같은 입력 재생

DB +1 재실행
11 → 12 (오류)

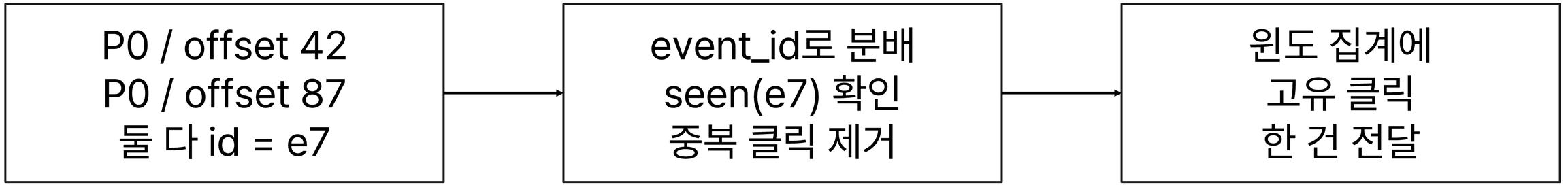
순서만 바꾸면 유실 위험으로 바뀐다. 원자적인 연결 또는 안전한 재적용이 필요하다.

두 종류의 중복

같은 로그 레코드의 재실행



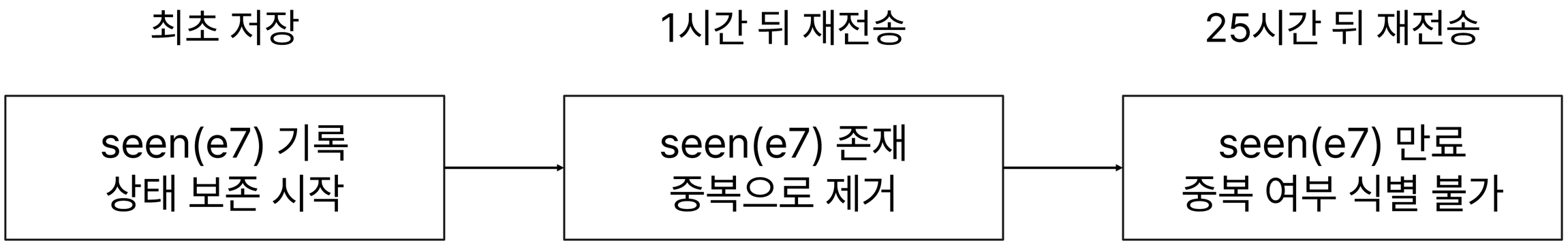
같은 클릭이 서로 다른 로그 위치에 기록됨



Exactly-once만으로 서로 다른 로그 위치에 기록된 동일 클릭이 합쳐지지는 않는다.

중복 제거 상태와 TTL

예제: 최초 저장 후 24시간 보존하고, 재전송으로 TTL을 늘리지 않는다.



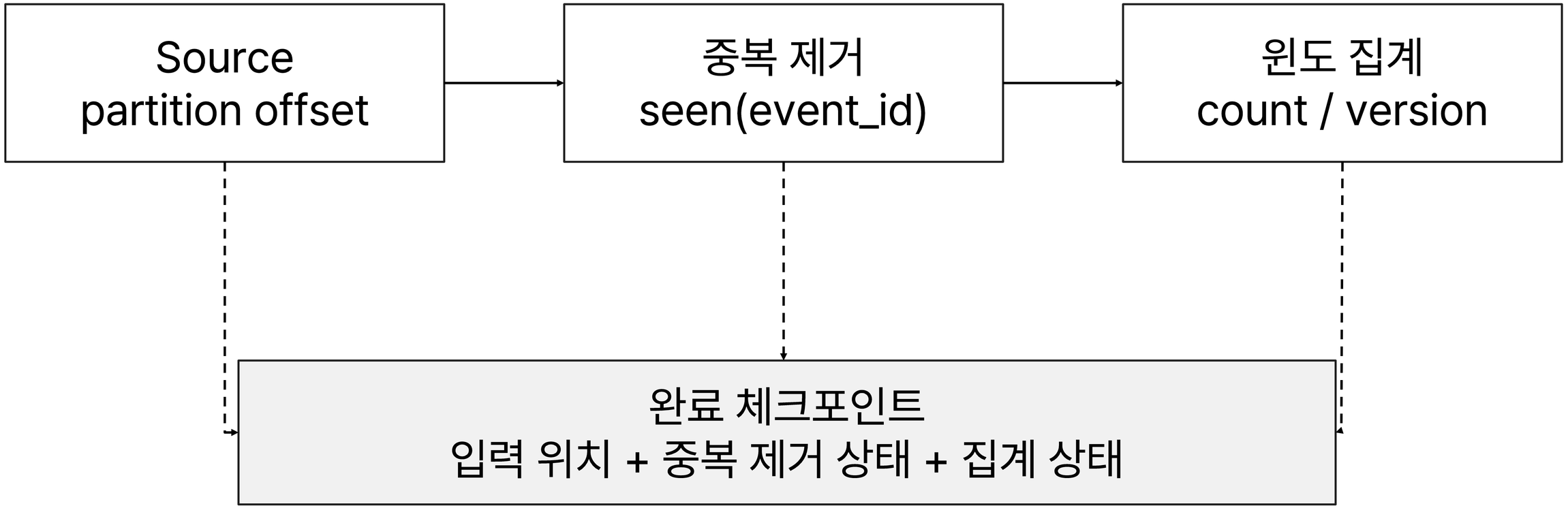
24시간 상태의 최소 비용

10억 ID × 32 B = 32 GB

상태도 함께 복구한다. 만료 후 입력의 집계 여부는 윈도 / 지연 처리 정책에 달려 있다.

체크포인트

입력 위치와 연산 상태를 일관된 복구 경계로 저장한다.



체크포인트는 복구 일관성을 다룬다. 워터마크의 시간 진행과 역할이 다르다.

복구와 재실행

관리 상태는 완료 지점으로 되돌리고 그 이후 입력을 재생한다.

단계	입력 위치	count
완료 체크포인트	42까지 처리	100
43~45 처리 후 장애	45까지 처리, 미확정	103
체크포인트 복원	다음 입력 43부터 재생	100
43~45 재실행 완료	45까지 처리	103

코드는 다시 실행될 수 있다. 외부 결과에도 한 번의 효과가 남도록 별도 계약이 필요하다.

외부 저장소의 쓰기 계약

현재 결과는 v7 / count 11이다. 각 행은 이 상태에 대한 독립 비교다.

들어온 결과	판정	DB의 결과
v6 / count 10	오래된 버전이므로 거부	v7 / count 11 유지
v7 / count 11	같은 내용의 재시도	변화 없음
v7 / count 12	동일 버전의 내용 충돌	반영하지 않고 오류 확인
v8 / count 12	새 버전으로 값 교체	v8 / count 12

같은 결과 키 / 집계 세대 안에서 비교와 교체를 원자적으로 수행한다. 버전은 복구 중에도 일관되어야 한다.

부분 집계의 수정

누적 부분 합계가 다시 오면 해당 부분값을 교체한다.

부분 키	이전 값	새 값	전체 합계 반영
salt 0	v1 = 10	v2 = 11	+1
salt 1	v1 = 7	변경 없음	0
전체	17	18	11을 다시 더하지 않음

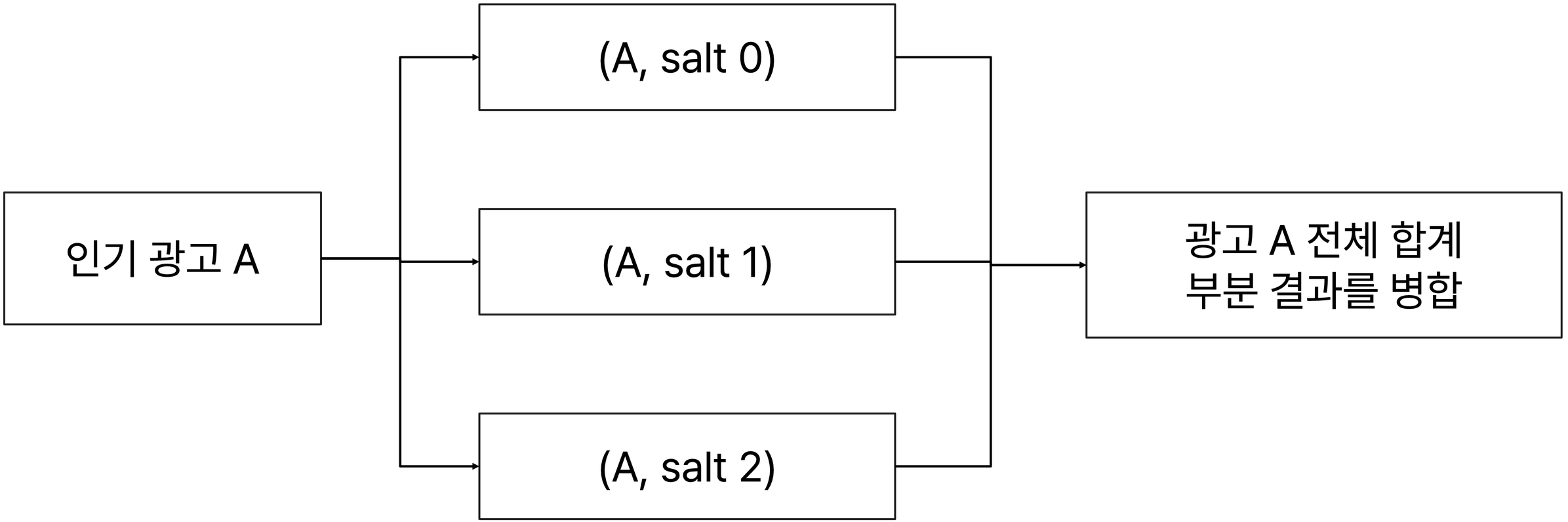
부분 키별 최신 값과 버전도 상태로 보존하고 함께 복구한다.

처리량과 병목

- 평균보다 피크를 기준으로
  - 평균 11,574건/s, 5배 피크 57,870건/s
  - 1 KB/건이면 피크 입력 약 57.9 MB/s
- 단계별 한계는 서로 다름
  - 10,000건/s 샤드라면 전체 최소 6개
  - 상태 접근 / 셔플 / sink 쓰기량을 각각 측정

인기 광고의 핫키

한 광고에 23,148건/초가 집중되면 키 자체를 나누어야 한다.



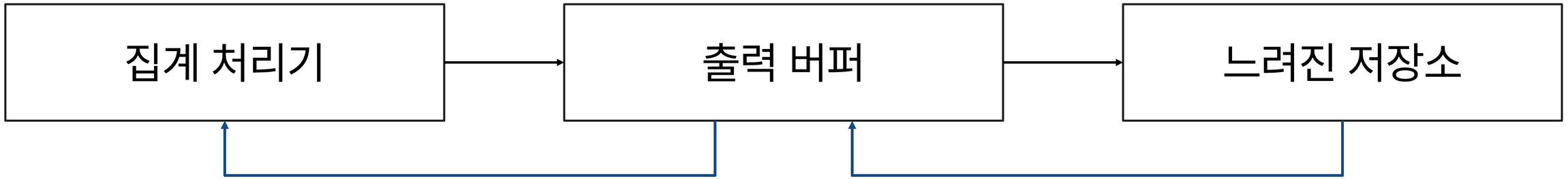
salt = hash(event_id) mod S. 최종 단계에는 매 클릭 대신 줄어든 부분 결과를 보낸다.

병렬성의 한계

- 작업자 수만 늘려서는 부족
  - 입력 파티션 수가 소스 읽기 병렬성을 제한
  - 단일 키와 최종 병합은 별도 병목이 됨
- 상태와 순위의 정확성 유지
  - 재분배 때 상태 / salt 매핑의 일관성 확보
  - 광고별 전체 합계가 완성된 뒤 Top-K 계산

장애와 역압력

저장소가 느려지면 출력 버퍼가 차고 상류 처리도 대기한다.



검정: 데이터 전달　　파랑: 처리 대기가 상류로 전파

처리율이 유입률보다 커야 적체가 줄어든다

예시: 120,000건 ÷ (12,000 − 10,000)건/초 = 60초

이상적인 해소 시간 = 적체 B ÷ (처리율 μ − 유입률 λ). 처리율이 변하면 다시 계산한다.

운영 지표

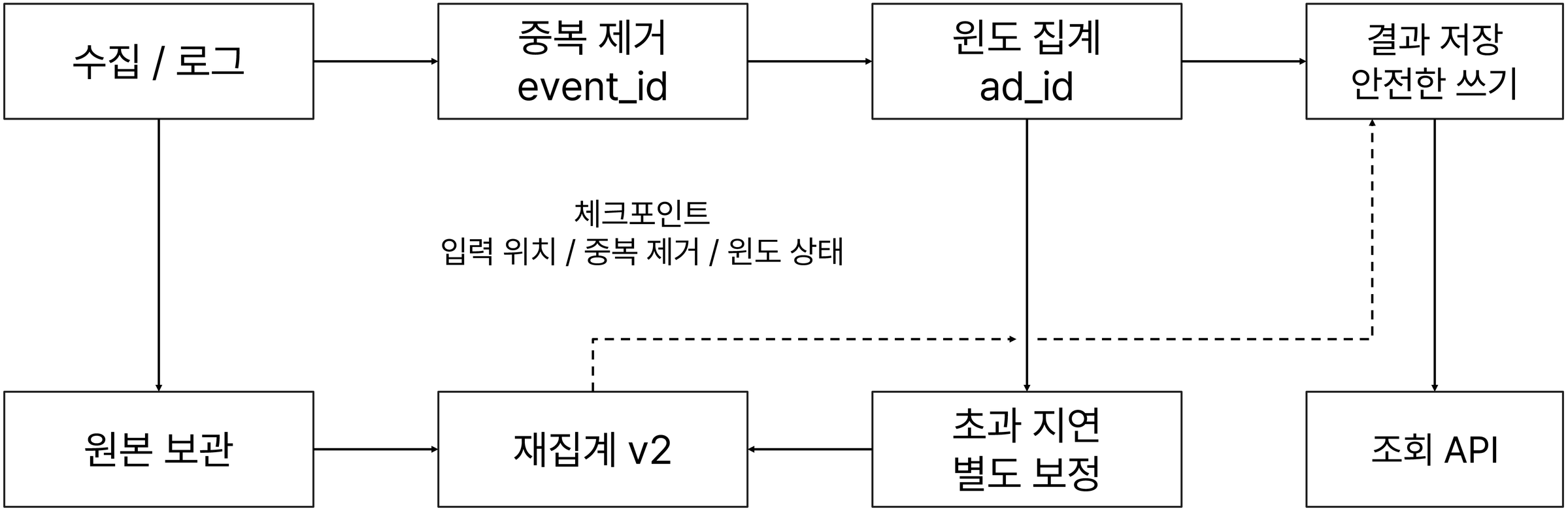
- 사용자가 보는 지연
  - 수집 → 조회 가능까지 p95 / p99 지연
  - 워터마크 지연 / 늦은 이벤트 / 수정 비율
- 복구와 정확성의 신호
  - 입력 파티션별 lag / 핫키 부하 / 상태 크기
  - 체크포인트 실패 / sink 재시도 / 대사 차이

집계 결과 검증

- 독립된 기준 집계와 비교
  - 원본에서 event_id 중복 제거 후 다시 계산
  - 광고 / 윈도 / 규칙 버전별 count를 대사
- 실패 경계에서 성질 검증
  - 중복 / 순서 변경 / 지연 입력에도 기대값 유지
  - DB 쓰기 직후 장애와 오래된 버전 재시도

장애와 보정을 포함한 구조

복구 상태와 과거 보정 경로까지 포함한 설계



재집계는 별도 결과 버전에 기록한다. 검증 후 조회 버전을 전환한다.

설계 선택과 비용

요구사항이 바뀌면 얻는 효과와 비용을 함께 설명한다.

선택	얻는 효과	늘어나는 비용 / 조건
허용 지연 확대	온라인 보정 범위 확대	상태 보관 / 결과 수정
조기 결과 출력	낮은 관측 지연	추가 쓰기 / 잠정값 계약
차원 사전 집계	자주 쓰는 조회 가속	상태 수 / 저장 행 수
핫키 분할	집중 부하 분산	부분 결과의 병합
원본 장기 보관	오래된 오류 재계산	보관 / 스캔 / 재생 비용

완전성 / 신선도 / 비용을 각각 측정해야 선택의 효과를 확인할 수 있다.

면접에서 설명하는 순서

- 입력과 출력의 계약부터 시작한다
  - 클릭의 단위, 조회 구간, 필터와 결과의 잠정 / 확정 기준을 정한다.
  - 처리량과 보존량을 계산해 로그 / 상태 / 저장소의 규모를 잡는다.
- 정상 동작 뒤에 실패 상황을 붙인다
  - 수집부터 집계 / 조회까지 흐름을 그리고 지연 / 중복 / 장애를 대입한다.
  - 인기 광고의 편향과 재집계가 생겨도 계약이 유지되는지 확인한다.

핵심 정리

- 집계 결과의 의미를 먼저 정한다
  - 어떤 클릭을 어느 시간 구간에 세는지 데이터 계약으로 고정한다.
- 시간과 장애는 별도로 다룬다
  - 워터마크는 이벤트 시간 진행, 체크포인트는 복구 일관성을 다룬다.
- 결과가 한 번 반영되는 경로를 만든다
  - 업무 중복 제거와 외부 저장소의 쓰기 계약을 함께 설계한다.
- 과거 결과를 고칠 수 있어야 한다
  - 원본 보관 / 버전 있는 재집계 / 대조 검증을 운영 구조에 포함한다.

토론 1. 신선도와 완전성

- 상황
  - 대시보드는 클릭 후 5초 안에 갱신되어야 한다.
  - 전체 클릭의 1%는 최대 10분 늦게 도착한다.
- 설계 질문
  - 5초 신선도와 누락 없는 집계를 어떤 결과 계약으로 제공할까?
  - 워터마크 / 허용 지연 / 확정 시점 / 보정 경로를 어떻게 정할까?
- 토론할 기준
  - 평균 지연만 보지 말고 잠정값과 확정값의 의미를 설명해 보자.

토론 2. 인기 광고의 편향

- 상황
  - 피크 입력의 40%가 하나의 광고에서 발생한다.
  - 최근 5분의 정확한 Top-100을 매분 제공해야 한다.
- 설계 질문
  - 광고 ID만으로 나누면 어느 연산자가 먼저 병목이 될까?
  - Salt로 나눈 부분 집계와 Top-100을 어떤 순서로 계산할까?
- 토론할 기준
  - 샤드 수 / 병합 빈도 / 정확성 / 지연의 관계를 설명해 보자.

토론 3. 무중단 재집계

- 상황
  - 집계 규칙 버그로 지난 6시간의 클릭 수가 잘못되었다.
  - 실시간 입력과 광고주 조회는 계속 운영해야 한다.
- 설계 질문
  - 원본을 다시 읽는 동안 실시간 입력과의 경계를 어떻게 정할까?
  - 검증 / 결과 교체 / 롤백을 어떤 단위와 순서로 수행할까?
- 토론할 기준
  - 같은 클릭이 두 결과에 중복 포함되거나 경계에서 빠지지 않아야 한다.

토론 1. 해설

- 동시에 만족할 수 있는 계약
  - 도착하지 않은 클릭까지 발생 후 5초 내 반영할 수는 없다.
  - 수신한 이벤트의 잠정치와 늦은 이벤트를 포함한 수정치를 제공한다.
- 구체적인 처리 정책
  - 지연 분포로 허용 범위를 정하고 범위 밖 이벤트는 별도로 보정한다.
  - 정산 확정 시점은 최장 지연 / 업무 규칙 / 비용을 함께 고려한다.
- 운영에서 확인할 점
  - 갱신 지연의 백분위와 수정 비율을 서로 다른 지표로 측정한다.

토론 2. 해설

- 키를 나눈 뒤 전체 클릭 수를 복원한다
  - (ad_id, salt, window)로 부분 집계하고 광고별로 다시 합친다.
  - 완성된 광고별 총합을 대상으로 Top-100 후보를 추린다.
- 병합 비용도 제어해야 한다
  - 매 클릭 대신 일정 간격의 부분 집계 업데이트를 전송한다.
  - 수정된 부분값은 교체하고, 이전 전체 합계에 반복해서 더하지 않는다.
- 검증할 조건
  - 같은 윈도 / 집계 버전 / 동점 규칙으로 순위를 계산한다.

토론 3. 해설

- 새 결과를 만들 경계를 고정한다
  - 원본 스냅샷 또는 입력 위치 H와 새 규칙 버전 v2를 기록한다.
  - v2 전용 결과에 H까지 재집계하고 이후 입력을 이어 반영한다.
- 검증한 결과만 조회에 공개한다
  - 광고 / 시간별 대조와 지연 이벤트 검증 후 활성 버전을 전환한다.
  - 실시간과 재집계 자원을 나누고 v1을 유지해 롤백한다.
- 핵심 실패 사례
  - 기존 count에 재집계 count를 더하면 같은 클릭을 다시 세게 된다.